
Special Topics in AI

Lecture 3: Transformer

University of Mazandaran

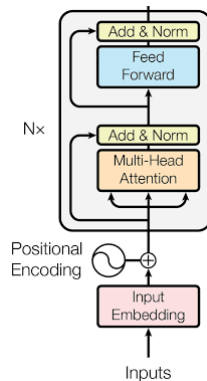
Fall 1405

- ▶ Encoder-decoder model based on (self-)attention mechanism
 - ▶ without any recurrence and convolutions
 - ▶ referred to as NLP's ImageNet moment
 - ▶ completely changed the landscape of deep learning models!
- ▶ Important concepts:
 - ▶ tokens
 - ▶ attention
 - ▶ positional encoding

¹Attention Is All You Need, Vaswani et al. NeurIPS (2017)

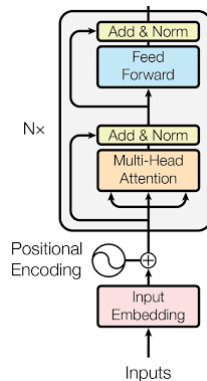
Transformer Encoder

- The encoder consists of $N=6$ identical layers



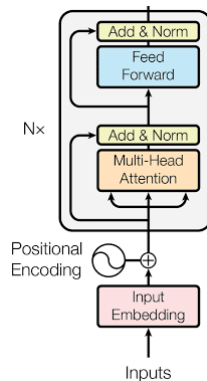
Transformer Encoder

- ▶ The encoder consists of $N=6$ identical layers
 - ▶ each layer has 2 sub-layers: multi-head attention and fully connected feed-forward network



Transformer Encoder

- ▶ The encoder consists of $N=6$ identical layers
 - ▶ each layer has 2 sub-layers: multi-head attention and fully connected feed-forward network
- ▶ Each sub-layer has a residual connection around it followed by layer normalization



Positional Encoding

- ▶ Unlike RNNs, transformers analyze all tokens in parallel
- ▶ But, self-attention has no built-in order
 - ▶ sees the sentence mostly as a set of token embeddings $\{\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2\}$
 - ▶ sentences with identical words, could mean opposite things!

The boy eats the fish $\rightarrow$ The fish eats the boy

Positional Encoding

- ▶ Positional encoding adds a position vector to each token
 - ▶ for every token position i , create a positional vector $\mathbf{p}_i$ with the same dimension as the token embedding $\mathbf{x}_i$ and add them to get $\mathbf{z}_i = \mathbf{x}_i + \mathbf{p}_i$

Positional Encoding

- ▶ Positional encoding adds a position vector to each token
 - ▶ for every token position i , create a positional vector $\mathbf{p}_i$ with the same dimension as the token embedding $\mathbf{x}_i$ and add them to get $\mathbf{z}_i = \mathbf{x}_i + \mathbf{p}_i$
 - ▶ for each position in the sequence (pos) and each dimension in the PE vector (i):

$$\text{even-indexed dimensions : } PE_{pos,2i} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

$$\text{odd-indexed dimensions : } PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

Positional Encoding: Example

Suppose the input sentence has embedded tokens $[\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3]$ and $d_{\text{model}} = 8$, i.e. $\mathbf{x}_i \in \mathbb{R}^8$. For $\mathbf{x}_2$:

$$pos = 2$$

$$PE_{pos} = [PE_{pos,0}, PE_{pos,1}, \dots, PE_{pos,7}]$$

$$PE_{pos,0} = \sin\left(\frac{2}{10000^{\frac{0}{8}}}\right), \quad PE_{pos,1} = \cos\left(\frac{2}{10000^{\frac{0}{8}}}\right)$$

$$PE_{pos,2} = \sin\left(\frac{2}{10000^{\frac{2}{8}}}\right), \quad PE_{pos,3} = \cos\left(\frac{2}{10000^{\frac{2}{8}}}\right)$$

and so on

Positional Encoding

- ▶ Positional encoding adds a position vector to each token
 - ▶ for every token position i , create a positional vector $\mathbf{p}_i$ with the same dimension as the token embedding $\mathbf{x}_i$ and add them to get $\mathbf{z}_i = \mathbf{x}_i + \mathbf{p}_i$

$$[\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2] \rightarrow [\mathbf{z}_0 = \mathbf{x}_0 + \mathbf{p}_0, \mathbf{z}_1 = \mathbf{x}_1 + \mathbf{p}_1, \mathbf{z}_2 = \mathbf{x}_2 + \mathbf{p}_2]$$

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{Z}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{Z}\mathbf{W}_V$$

$$\mathbf{Z} = [\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n]$$

Positional Encoding

- ▶ Positional encoding adds a position vector to each token
 - ▶ for every token position i , create a positional vector $\mathbf{p}_i$ with the same dimension as the token embedding $\mathbf{x}_i$ and add them to get $\mathbf{z}_i = \mathbf{x}_i + \mathbf{p}_i$

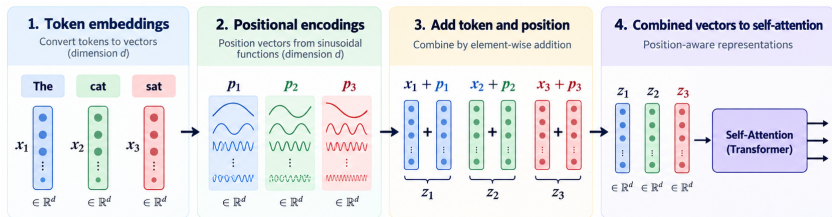
$$[\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2] \rightarrow [\mathbf{z}_0 = \mathbf{x}_0 + \mathbf{p}_0, \mathbf{z}_1 = \mathbf{x}_1 + \mathbf{p}_1, \mathbf{z}_2 = \mathbf{x}_2 + \mathbf{p}_2]$$

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{Z}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{Z}\mathbf{W}_V$$

$$\mathbf{Z} = [\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n]$$

Positional encoding tells the transformer (i) what each token is, and (ii) where it is positioned.

Transformer Encoder



$$z_i = x_i + p_i \quad (\text{element-wise addition})$$

Without positional encoding

Tokens are an unordered bag (same vectors regardless of position).



Set / bag of tokens
(no order)

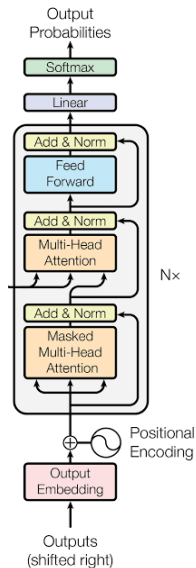
With positional encoding

Tokens are placed on an ordered timeline.



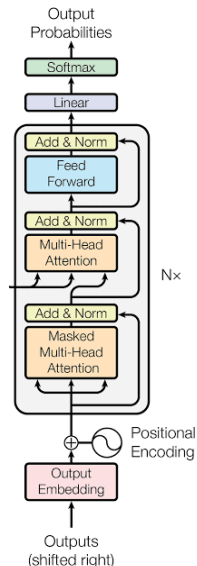
Transformer Decoder

- The decoder also has $N=6$ identical layers



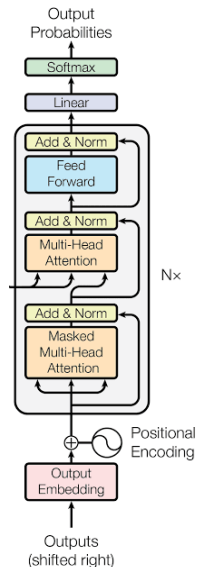
Transformer Decoder

- ▶ The decoder also has $N=6$ identical layers
 - ▶ decoder layer is identical to the encoder layer with an additional 3rd sub-layer
 - ▶ the sub-layer performs multi-head attention over the output of the encoder



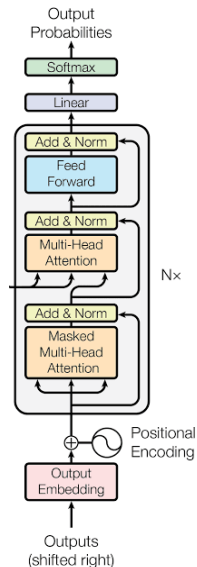
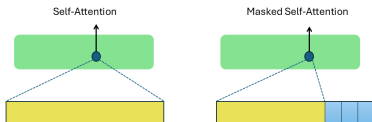
Transformer Decoder

- ▶ The decoder also has $N=6$ identical layers
 - ▶ decoder layer is identical to the encoder layer with an additional 3rd sub-layer
 - ▶ the sub-layer performs multi-head attention over the output of the encoder
- ▶ The masked self-attention sub-layer prevents positions from attending to unseen positions
 - ▶ the prediction for position i can depend only on the known outputs at positions $< i$



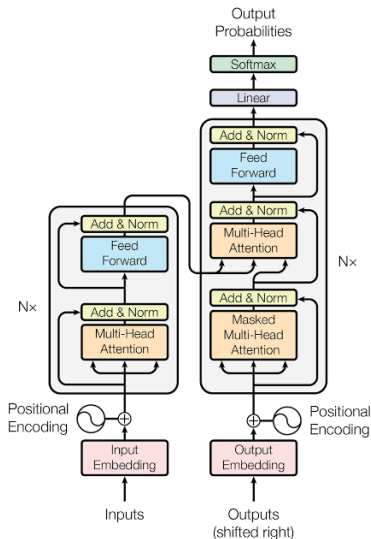
Transformer Decoder

- ▶ The decoder also has $N=6$ identical layers
 - ▶ decoder layer is identical to the encoder layer with an additional 3rd sub-layer
 - ▶ the sub-layer performs multi-head attention over the output of the encoder
- ▶ The masked self-attention sub-layer prevents positions from attending to unseen positions
 - ▶ the prediction for position i can depend only on the known outputs at positions $< i$



Transformer Decoder

The full model



Coding a Transformer (PyTorch): init

```
class Transformer(nn.Module):  
    def __init__(self,  
        d_model: int = 512, # number of expected features in the encoder/decoder inputs  
        nhead: int = 8,     # number of heads in the multihead-attention models  
        num_encoder_layers: int = 6, # number of sub-encoder layers in encoder  
        num_decoder_layers: int = 6, # number of sub-decoder layers in the decoder  
        dim_feedforward: int = 2048 # dimension of the feedforward network model  
    ):
```

Coding a Transformer (PyTorch): init

```
class Transformer(nn.Module):
    def __init__(self,
        d_model: int = 512, # number of expected features in the encoder/decoder inputs
        nhead: int = 8,     # number of heads in the multihead-attention models
        num_encoder_layers: int = 6, # number of sub-encoder layers in encoder
        num_decoder_layers: int = 6, # number of sub-decoder layers in the decoder
        dim_feedforward: int = 2048 # dimension of the feedforward network model
    ):

        enc_layer = TransformerEncoderLayer(d_model, nhead, dim_feedforward, ...)
        enc_norm = LayerNorm(d_model, eps=layer_norm_eps, **factory_kwargs)
```

Coding a Transformer (PyTorch): init

```
class Transformer(nn.Module):
    def __init__(self,
        d_model: int = 512, # number of expected features in the encoder/decoder inputs
        nhead: int = 8,     # number of heads in the multihead-attention models
        num_encoder_layers: int = 6, # number of sub-encoder layers in encoder
        num_decoder_layers: int = 6, # number of sub-decoder layers in the decoder
        dim_feedforward: int = 2048 # dimension of the feedforward network model
    ):

        enc_layer = TransformerEncoderLayer(d_model, nhead, dim_feedforward, ...)
        enc_norm = LayerNorm(d_model, eps=layer_norm_eps, **factory_kwargs)

        dec_layer = TransformerDecoderLayer(d_model, nhead, dim_feedforward, ...)
        dec_norm = LayerNorm(d_model, eps=layer_norm_eps, **factory_kwargs)
```

Coding a Transformer (PyTorch): init

```
class Transformer(nn.Module):
    def __init__(self,
        d_model: int = 512, # number of expected features in the encoder/decoder inputs
        nhead: int = 8,     # number of heads in the multihead-attention models
        num_encoder_layers: int = 6, # number of sub-encoder layers in encoder
        num_decoder_layers: int = 6, # number of sub-decoder layers in the decoder
        dim_feedforward: int = 2048 # dimension of the feedforward network model
    ):

        enc_layer = TransformerEncoderLayer(d_model, nhead, dim_feedforward, ...)
        enc_norm = LayerNorm(d_model, eps=layer_norm_eps, **factory_kwargs)

        dec_layer = TransformerDecoderLayer(d_model, nhead, dim_feedforward, ...)
        dec_norm = LayerNorm(d_model, eps=layer_norm_eps, **factory_kwargs)

        self.encoder = TransformerEncoder(enc_layer, num_encoder_layers, enc_norm)
        self.decoder = TransformerDencoder(dec_layer, num_decoder_layers, dec_norm)
```

Coding a Transformer (PyTorch): forward pass

```
def forward(self,
    src: Tensor,      # the sequence to the encoder (required)
    tgt: Tensor,      # the sequence to the decoder - target (required)
    src_mask: Optional[Tensor]=None, # the additive mask for the src sequence (opt)
    tgt_mask: Optional[Tensor]=None, # the additive mask for the tgt sequence (opt)
):
```

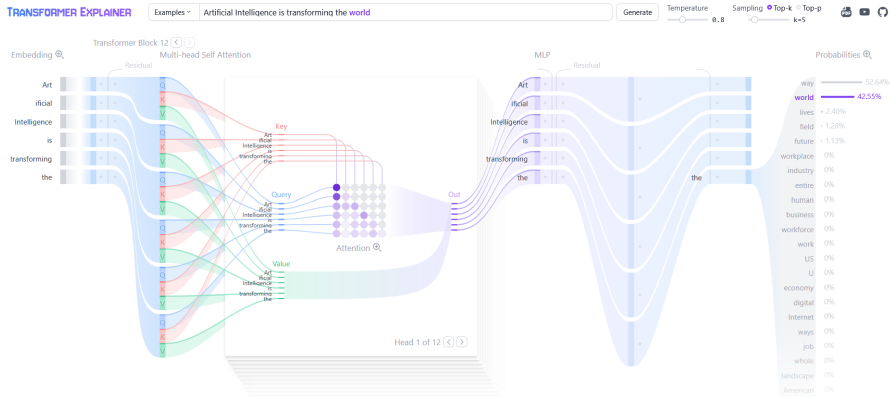
Coding a Transformer (PyTorch): forward pass

```
def forward(self,
    src: Tensor,      # the sequence to the encoder (required)
    tgt: Tensor,      # the sequence to the decoder - target (required)
    src_mask: Optional[Tensor]=None, # the additive mask for the src sequence (opt)
    tgt_mask: Optional[Tensor]=None, # the additive mask for the tgt sequence (opt)
):
    memory = self.encoder(src, mask=src_mask, ...)

    output = self.decoder(tgt, memory, tgt_mask=tgt_mask, ...)

    return output
```

Visualization of Transformers



<https://poloclub.github.io/transformer-explainer/>

Pros:

- ▶ Transformer operates on data in parallel which accelerates the learning process, compared to RNN which operates sequentially
- ▶ Transformer can deal with long-term dependencies in sequences

Cons:

- ▶ Transformer scales quadratically with the number of inputs
- ▶ They are memory-intensive and requires lots of data and long training

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Attention is All You Need